

El problema de la satisfacibilidad booleana y el vacío de la intersección con lenguajes regulares

Jossué Arteché Muñoz

Facultad de Matemática y Computación
Universidad de La Habana

10 de junio del 2026

Tutores: MSc. Fernando Raul Rodriguez Flores
Lic. Raudel Alejandro Gómez Molina

Problema de la satisfacibilidad booleana (SAT)

- Determinar si una fórmula booleana es satisfacible.

Problema de la satisfacibilidad booleana (SAT)

- Determinar si una fórmula booleana es satisfacible.
- Primer problema demostrado NP-completo.

Problema de la satisfacibilidad booleana (SAT)

- Determinar si una fórmula booleana es satisfacible.
- Primer problema demostrado NP-completo.
- Aplicaciones:
 - Verificación formal de hardware y software.
 - Planificación y calendarización.
 - Inteligencia artificial y razonamiento automático.
 - Diseño y optimización de circuitos.

Problema de la satisfacibilidad booleana (SAT)

- Determinar si una fórmula booleana es satisfacible.
- Primer problema demostrado NP-completo.
- Aplicaciones:
 - Verificación formal de hardware y software.
 - Planificación y calendarización.
 - Inteligencia artificial y razonamiento automático.
 - Diseño y optimización de circuitos.
- Resolver SAT eficientemente implica demostrar $P = NP$.

Enfoque basado en lenguajes formales

- En MATCOM se estudian enfoques basados en lenguajes formales.

Enfoque basado en lenguajes formales

- En MATCOM se estudian enfoques basados en lenguajes formales.
- A partir de una instancia de SAT se construye un DFA.

Enfoque basado en lenguajes formales

- En MATCOM se estudian enfoques basados en lenguajes formales.
- A partir de una instancia de SAT se construye un DFA.
- Se hacen consistentes las interpretaciones con formalismos:

Gramáticas Libres de Contexto

- Se determina si es vacía la intersección.

Gramáticas Matriciales

Gramáticas de Concatenación de Rango

Enfoque basado en lenguajes formales

Si existiera un formalismo que:

Si existiera un formalismo que:

- Puede intersectarse eficientemente con un DFA.

Si existiera un formalismo que:

- Puede intersectarse eficientemente con un DFA.
- Tiene problema del vacío decidible en tiempo polinomial.

Si existiera un formalismo que:

- Puede intersectarse eficientemente con un DFA.
- Tiene problema del vacío decidible en tiempo polinomial.

Entonces SAT podría resolverse en tiempo polinomial.

Fundamental Study

On multiple context-free grammars*

Hiroyuki Seki

*Department of Information and Computer Sciences, Faculty of Engineering Science,
Osaka University, Toyonaka, Osaka 560, Japan*

Takashi Matsumura

Nomura Research Institute, Ltd., Tyu-o-ku, Tokyo 104, Japan

Mamoru Fujii

College of General Education, Osaka University, Toyonaka, Osaka, 560, Japan

Tadao Kasami

*Department of Information and Computer Sciences, Faculty of Engineering Science,
Osaka University, Toyonaka, Osaka 560, Japan*

Fundamental Study

On multiple context-free grammars*

Se definen las **gramáticas libres de contexto múltiple paralelas** (pMCFG).

Hiroyuki Seki

*Department of Information and Computer Sciences, Faculty of Engineering Science,
Osaka University, Toyonaka, Osaka 560, Japan*

Takashi Matsumura

Nomura Research Institute, Ltd., Tyu-o-ku, Tokyo 104, Japan

Mamoru Fujii

College of General Education, Osaka University, Toyonaka, Osaka, 560, Japan

Tadao Kasami

*Department of Information and Computer Sciences, Faculty of Engineering Science,
Osaka University, Toyonaka, Osaka 560, Japan*

Fundamental Study

On multiple context-free grammars*

Hiroyuki Seki

*Department of Information and Computer Sciences, Faculty of Engineering Science,
Osaka University, Toyonaka, Osaka 560, Japan*

Takashi Matsumura

Nomura Research Institute, Ltd., Tyy-o-ku, Tokyo 104, Japan

Mamoru Fujii

College of General Education, Osaka University, Toyonaka, Osaka, 560, Japan

Tadao Kasami

*Department of Information and Computer Sciences, Faculty of Engineering Science,
Osaka University, Toyonaka, Osaka 560, Japan*

Se definen las **gramáticas libres de contexto múltiple paralelas** (pMCFG).

- Se pueden intersectar con un DFA.

Fundamental Study

On multiple context-free grammars*

Hiroyuki Seki

*Department of Information and Computer Sciences, Faculty of Engineering Science,
Osaka University, Toyonaka, Osaka 560, Japan*

Takashi Matsumura

Nomura Research Institute, Ltd., Tyu-o-ku, Tokyo 104, Japan

Mamoru Fujii

College of General Education, Osaka University, Toyonaka, Osaka, 560, Japan

Tadao Kasami

*Department of Information and Computer Sciences, Faculty of Engineering Science,
Osaka University, Toyonaka, Osaka 560, Japan*

Se definen las **gramáticas libres de contexto múltiple paralelas** (pMCFG).

- Se pueden intersectar con un DFA.
- Tienen vacío decidible en tiempo polinomial.

Por tanto

Por tanto

$$P = NP$$

Por tanto

$$P = NP$$

...pero eso no es lo que vamos a ver hoy.

Objetivos del trabajo

Objetivos del trabajo

Objetivo 1

Formular un teorema que caracterice los formalismos capaces de describir interpretaciones satisfacibles.

Objetivos del trabajo

Objetivo 1

Formular un teorema que caracterice los formalismos capaces de describir interpretaciones satisfacibles.

Objetivo 2

Analizar críticamente las propiedades de las pMCFG propuestas en la literatura.

Estructura de la presentación

1 Preliminares

- Teoría de lenguajes
- Definición de SAT

2 Construcción del lenguaje de las interpretaciones satisfacibles

- Codificación de las interpretaciones como cadenas
- Autómata fórmula
- Recuperar la consistencia de las interpretaciones

3 Analizar las pMCFG

- **Alfabeto** (Σ): conjunto finito de símbolos.

$$\Sigma = \{a, b, c\}$$

- **Alfabeto** (Σ): conjunto finito de símbolos.

$$\Sigma = \{a, b, c\}$$

- **Cadena**: secuencia finita de símbolos de Σ .

$$abca \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Alfabeto** (Σ): conjunto finito de símbolos.

$$\Sigma = \{a, b, c\}$$

- **Cadena**: secuencia finita de símbolos de Σ .

$$abca \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Concatenación**:

$$\alpha = ab \text{ y } \beta = ca \implies \alpha\beta = abca$$

- **Alfabeto** (Σ): conjunto finito de símbolos.

$$\Sigma = \{a, b, c\}$$

- **Cadena**: secuencia finita de símbolos de Σ .

$$abca \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Concatenación**:

$$\alpha = ab \text{ y } \beta = ca \implies \alpha\beta = abca$$

- **Lenguaje**: conjunto de cadenas.

$$L = \{a^n b^n \mid n \geq 0\}$$

- **Alfabeto** (Σ): conjunto finito de símbolos.

$$\Sigma = \{a, b, c\}$$

- **Cadena**: secuencia finita de símbolos de Σ .

$$abca \in \Sigma^* \quad \varepsilon \text{ (cadena vacía)}$$

- **Concatenación**:

$$\alpha = ab \text{ y } \beta = ca \implies \alpha\beta = abca$$

- **Lenguaje**: conjunto de cadenas.

$$L = \{a^n b^n \mid n \geq 0\}$$

- **Formalismo**: mecanismo de algún tipo para describir lenguajes.

Definición

Una gramática es un formalismo que genera cadenas mediante la aplicación de reglas de producción.

Definición

Una gramática es un formalismo que genera cadenas mediante la aplicación de reglas de producción.

$$G = (N, \Sigma, P, S)$$

- N : no terminales
- Σ : terminales
- P : producciones
- S : símbolo inicial

Definición

Una gramática es un formalismo que genera cadenas mediante la aplicación de reglas de producción.

$$G = (N, \Sigma, P, S)$$

- N : no terminales
 - Σ : terminales
 - P : producciones
 - S : símbolo inicial
-

Ejemplo

$$S \rightarrow aS$$

$$S \Rightarrow aS \Rightarrow aaS \Rightarrow aab$$

$$S \rightarrow b$$

Definición

Una gramática es un formalismo que genera cadenas mediante la aplicación de reglas de producción.

$$G = (N, \Sigma, P, S)$$

- N : no terminales
 - Σ : terminales
 - P : producciones
 - S : símbolo inicial
-

Ejemplo

$$S \rightarrow aS$$

$$S \Rightarrow aS \Rightarrow aaS \Rightarrow aab$$

$$S \rightarrow b$$

$$L(A) = \{a^n b \mid n \geq 0\}.$$

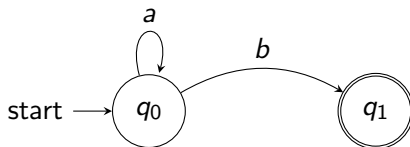
Autómatas Finitos Deterministas (DFA)

Autómatas Finitos Deterministas (DFA)

Definición. Un DFA es un modelo computacional que reconoce cadenas recorriendo estados de acuerdo con los símbolos leídos.

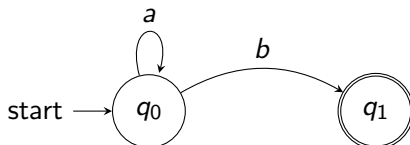
Autómatas Finitos Deterministas (DFA)

Definición. Un DFA es un modelo computacional que reconoce cadenas recorriendo estados de acuerdo con los símbolos leídos.



Autómatas Finitos Deterministas (DFA)

Definición. Un DFA es un modelo computacional que reconoce cadenas recorriendo estados de acuerdo con los símbolos leídos.

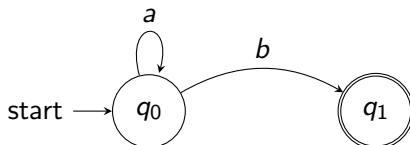


Cadenas aceptadas:

$b, ab, aab, aaab, \dots$

Autómatas Finitos Deterministas (DFA)

Definición. Un DFA es un modelo computacional que reconoce cadenas recorriendo estados de acuerdo con los símbolos leídos.



Cadenas aceptadas:

$b, ab, aab, aaab, \dots$

Por tanto,

$$L(A) = \{a^n b \mid n \geq 0\}.$$

Problemas de interés

Intersección con DFA

Dado una instancia de un formalismo G y un DFA $\mathcal{A}$, ¿es posible construir una nueva instancia del mismo formalismo que describa

$$L(G) \cap L(\mathcal{A})?$$

Intersección con DFA

Dado una instancia de un formalismo G y un DFA $\mathcal{A}$, ¿es posible construir una nueva instancia del mismo formalismo que describa

$$L(G) \cap L(\mathcal{A})?$$

Si la respuesta es sí, entonces el formalismo es **cerrado bajo intersección con DFA**.

Problemas de interés

Intersección con DFA

Dado una instancia de un formalismo G y un DFA $\mathcal{A}$, ¿es posible construir una nueva instancia del mismo formalismo que describa

$$L(G) \cap L(\mathcal{A})?$$

Si la respuesta es sí, entonces el formalismo es **cerrado bajo intersección con DFA**.

Problema del vacío

Dada una instancia de un formalismo G ,

$$L(G) = \emptyset?$$

Intersección con DFA

Dado una instancia de un formalismo G y un DFA $\mathcal{A}$, ¿es posible construir una nueva instancia del mismo formalismo que describa

$$L(G) \cap L(\mathcal{A})?$$

Si la respuesta es sí, entonces el formalismo es **cerrado bajo intersección con DFA**.

Problema del vacío

Dada una instancia de un formalismo G ,

$$L(G) = \emptyset?$$

Determinar si el lenguaje generado contiene al menos una cadena.

1 Preliminares

- Teoría de lenguajes
- Definición de SAT

2 Construcción del lenguaje de las interpretaciones satisfacibles

- Codificación de las interpretaciones como cadenas
- Autómata fórmula
- Recuperar la consistencia de las interpretaciones

3 Analizar las pMCFG

Problema de la satisfacibilidad booleana (SAT)

- **Variable booleana:** $x_1, x_2, x_3, \dots$ toman valores 1 (verdadero) o 0 (falso).

Problema de la satisfacibilidad booleana (SAT)

- **Variable booleana:** $x_1, x_2, x_3, \dots$ toman valores 1 (verdadero) o 0 (falso).
- **Literal:** $x_i, \neg x_i$ (la variable o su negación).

Problema de la satisfacibilidad booleana (SAT)

- **Variable booleana:** $x_1, x_2, x_3, \dots$ toman valores 1 (verdadero) o 0 (falso).
- **Literal:** $x_i, \neg x_i$ (la variable o su negación).
- **Cláusula:** Disyunción de literales.

$$(x_1 \vee \neg x_2 \vee x_3)$$

Problema de la satisfacibilidad booleana (SAT)

- **Variable booleana:** $x_1, x_2, x_3, \dots$ toman valores 1 (verdadero) o 0 (falso).
- **Literal:** $x_i, \neg x_i$ (la variable o su negación).
- **Cláusula:** Disyunción de literales.

$$(x_1 \vee \neg x_2 \vee x_3)$$

- **Forma Normal Conjuntiva (FNC)**

Conjunción de cláusulas.

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

Problema de la satisfacibilidad booleana (SAT)

- **Variable booleana:** $x_1, x_2, x_3, \dots$ toman valores 1 (verdadero) o 0 (falso).
- **Literal:** $x_i, \neg x_i$ (la variable o su negación).
- **Cláusula:** Disyunción de literales.

$$(x_1 \vee \neg x_2 \vee x_3)$$

- **Forma Normal Conjuntiva (FNC)**

Conjunción de cláusulas.

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

Problema SAT

¿Existe una asignación de valores de verdad para las variables que haga verdadera toda la fórmula?

1 Preliminares

- Teoría de lenguajes
- Definición de SAT

2 Construcción del lenguaje de las interpretaciones satisfacibles

- Codificación de las interpretaciones como cadenas
- Autómata fórmula
- Recuperar la consistencia de las interpretaciones

3 Analizar las pMCFG

Codificación de interpretaciones como cadenas

Idea

Codificación de interpretaciones como cadenas

Idea

Cada asignación de variables se representa como una cadena.

Codificación de interpretaciones como cadenas

Idea

Cada asignación de variables se representa como una cadena.

Regla de codificación

- Si la variable $x_i = 1$, el símbolo i es 1.

Codificación de interpretaciones como cadenas

Idea

Cada asignación de variables se representa como una cadena.

Regla de codificación

- Si la variable $x_i = 1$, el símbolo i es 1.
- Si $x_i = 0$, el símbolo i es 0.

Codificación de interpretaciones como cadenas

Idea

Cada asignación de variables se representa como una cadena.

Regla de codificación

- Si la variable $x_i = 1$, el símbolo i es 1.
- Si $x_i = 0$, el símbolo i es 0.
- Se añade el símbolo final d .

Codificación de interpretaciones como cadenas

Idea

Cada asignación de variables se representa como una cadena.

Regla de codificación

- Si la variable $x_i = 1$, el símbolo i es 1.
- Si $x_i = 0$, el símbolo i es 0.
- Se añade el símbolo final d .
- Se repite la cadena por cada cláusula.

Codificación de interpretaciones como cadenas

Idea

Cada asignación de variables se representa como una cadena.

Regla de codificación

- Si la variable $x_i = 1$, el símbolo i es 1.
- Si $x_i = 0$, el símbolo i es 0.
- Se añade el símbolo final d .
- Se repite la cadena por cada cláusula.

Ejemplo (4 variables, 2 cláusulas)

Codificación de interpretaciones como cadenas

Idea

Cada asignación de variables se representa como una cadena.

Regla de codificación

- Si la variable $x_i = 1$, el símbolo i es 1.
- Si $x_i = 0$, el símbolo i es 0.
- Se añade el símbolo final d .
- Se repite la cadena por cada cláusula.

Ejemplo (4 variables, 2 cláusulas)

Asignación:

$$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0$$

Codificación de interpretaciones como cadenas

Idea

Cada asignación de variables se representa como una cadena.

Regla de codificación

- Si la variable $x_i = 1$, el símbolo i es 1.
- Si $x_i = 0$, el símbolo i es 0.
- Se añade el símbolo final d .
- Se repite la cadena por cada cláusula.

Ejemplo (4 variables, 2 cláusulas)

Asignación:

Cadena base:

$$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0$$

1010

Codificación de interpretaciones como cadenas

Idea

Cada asignación de variables se representa como una cadena.

Regla de codificación

- Si la variable $x_i = 1$, el símbolo i es 1.
- Si $x_i = 0$, el símbolo i es 0.
- Se añade el símbolo final d .
- Se repite la cadena por cada cláusula.

Ejemplo (4 variables, 2 cláusulas)

Asignación:

Cadena base:

$$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0$$

1010

Con separador:

1010 d

Codificación de interpretaciones como cadenas

Idea

Cada asignación de variables se representa como una cadena.

Regla de codificación

- Si la variable $x_i = 1$, el símbolo i es 1.
- Si $x_i = 0$, el símbolo i es 0.
- Se añade el símbolo final d .
- Se repite la cadena por cada cláusula.

Ejemplo (4 variables, 2 cláusulas)

Asignación:

$$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0$$

Con separador:

1010*d*

Cadena base:

1010

Como hay 2 cláusulas:

1010*d* 1010*d*

Lenguaje de interpretaciones satisfacibles

Idea

El conjunto de interpretaciones de una fórmula se puede representar como un lenguaje.

Idea

El conjunto de interpretaciones de una fórmula se puede representar como un lenguaje.

Definición intuitiva

$$L_{\text{sat}}F = \{ w \mid w \text{ codifica una interpretación que satisface } F \}$$

Idea

El conjunto de interpretaciones de una fórmula se puede representar como un lenguaje.

Definición intuitiva

$$L_{\text{sat}}F = \{ w \mid w \text{ codifica una interpretación que satisface } F \}$$

Conexión con SAT

$$F \in SAT \iff L_{\text{sat}}F \neq \emptyset$$

1 Preliminares

- Teoría de lenguajes
- Definición de SAT

2 Construcción del lenguaje de las interpretaciones satisfacibles

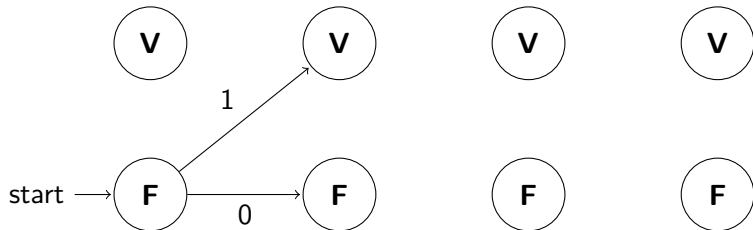
- Codificación de las interpretaciones como cadenas
- Autómata fórmula
- Recuperar la consistencia de las interpretaciones

3 Analizar las pMCFG

Lenguaje de una cláusula:

$L_C = \{w \mid w \text{ codifica una interpretación que satisface } C\}$

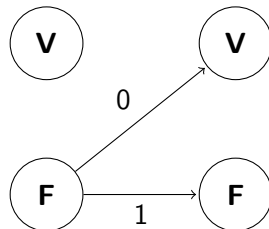
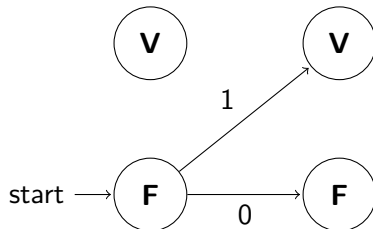
Cláusula: $(x_1 \vee \neg x_3)$ puede verse como $(x_1 \vee \cancel{x_2} \vee \neg x_3)$



Lenguaje de una cláusula:

$L_C = \{w \mid w \text{ codifica una interpretación que satisface } C\}$

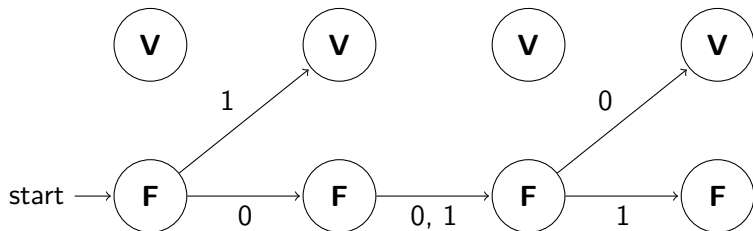
Cláusula: $(x_1 \vee \neg x_3)$ puede verse como $(x_1 \vee \cancel{x_2} \vee \neg x_3)$



Lenguaje de una cláusula:

$L_C = \{w \mid w \text{ codifica una interpretación que satisface } C\}$

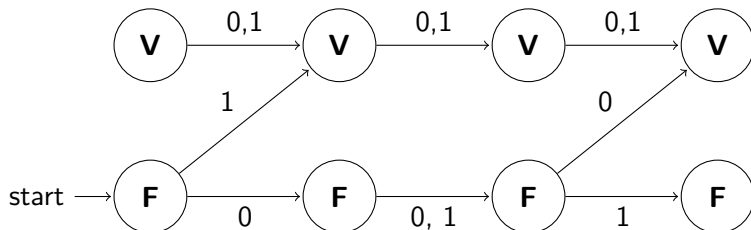
Cláusula: $(x_1 \vee \neg x_3)$ puede verse como $(x_1 \vee \cancel{x_2} \vee \neg x_3)$



Lenguaje de una cláusula:

$L_C = \{w \mid w \text{ codifica una interpretación que satisface } C\}$

Cláusula: $(x_1 \vee \neg x_3)$ puede verse como $(x_1 \vee \cancel{x_2} \vee \neg x_3)$

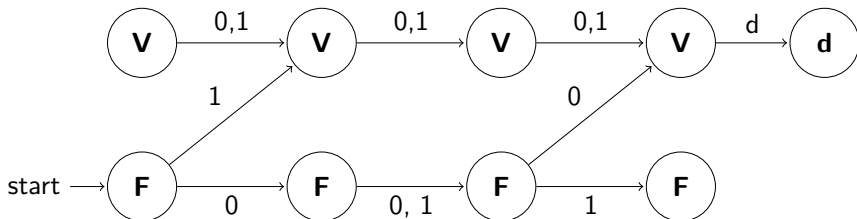


Autómata cláusula con separador

Lenguaje de una cláusula con separador:

$L_C = \{wd \mid w \text{ codifica una interpretación que satisface } C\}$

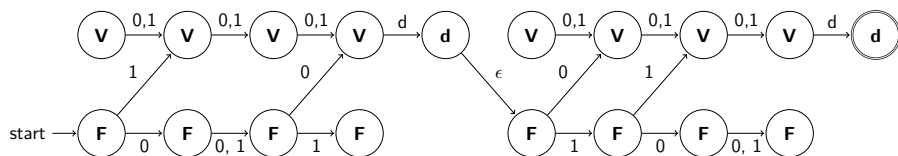
Cláusula: $(x_1 \vee \neg x_3)$ puede verse como $(x_1 \vee \cancel{x_2} \vee \neg x_3)$



Lenguaje de interpretaciones independientes:

$L_{\text{ind}}(F) = \{w_1dw_2d \dots w_md \mid w_i \text{ es una interpretación que satisface } C_i\}$

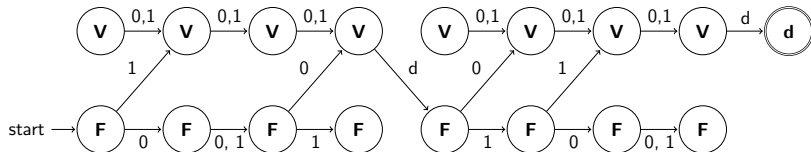
Fórmula: $(x_1 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2) \rightsquigarrow (x_1 \vee \cancel{x_2} \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \cancel{x_3})$



Lenguaje de interpretaciones independientes:

$L_{\text{ind}}(F) = \{w_1dw_2d \dots w_md \mid w_i \text{ es una interpretación que satisface } C_i\}$

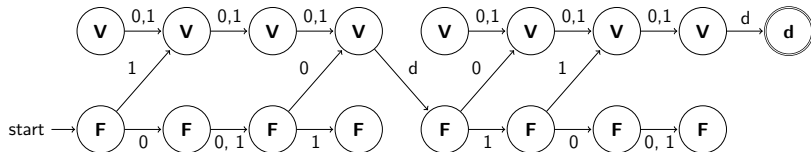
Fórmula: $(x_1 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2) \rightsquigarrow (x_1 \vee \cancel{x_2} \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \cancel{x_3})$



Lenguaje de interpretaciones independientes:

$L_{\text{ind}}(F) = \{w_1 d w_2 d \dots w_m d \mid w_i \text{ es una interpretación que satisface } C_i\}$

Fórmula: $(x_1 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2) \rightsquigarrow (x_1 \vee \cancel{x_2} \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \cancel{x_3})$



Autómata fórmula $\mathcal{A}_F$

$$L(\mathcal{A}_F) = L_{\text{ind}}(F)$$

1 Preliminares

- Teoría de lenguajes
- Definición de SAT

2 Construcción del lenguaje de las interpretaciones satisfacibles

- Codificación de las interpretaciones como cadenas
- Autómata fórmula
- Recuperar la consistencia de las interpretaciones

3 Analizar las pMCFG

Lenguaje de las repeticiones

Lenguaje de las repeticiones

Se necesita obtener el lenguaje de interpretaciones satisfacibles:

$$L_{\text{sat}}(F) = \{ w \mid w \text{ es una interpretación que satisface } F \}$$

Lenguaje de las repeticiones

Se necesita obtener el lenguaje de interpretaciones satisfacibles:

$$L_{\text{sat}}(F) = \{ w \mid w \text{ es una interpretación que satisface } F \}$$

Se dispone de un autómata $\mathcal{A}_F$ que reconoce:

$$L_{\text{ind}}(F) = \{ w_1 d w_2 d \dots w_m d \mid w_i \text{ satisface } C_i \}$$

Lenguaje de las repeticiones

Se necesita obtener el lenguaje de interpretaciones satisfacibles:

$$L_{\text{sat}}(F) = \{ w \mid w \text{ es una interpretación que satisface } F \}$$

Se dispone de un autómata $\mathcal{A}_F$ que reconoce:

$$L_{\text{ind}}(F) = \{ w_1 d w_2 d \dots w_m d \mid w_i \text{ satisface } C_i \}$$

Para forzar que todas las interpretaciones coincidan:

$$w_1 = w_2 = \dots = w_m = w$$

Lenguaje de las repeticiones

Se necesita obtener el lenguaje de interpretaciones satisfacibles:

$$L_{\text{sat}}(F) = \{ w \mid w \text{ es una interpretación que satisface } F \}$$

Se dispone de un autómata $\mathcal{A}_F$ que reconoce:

$$L_{\text{ind}}(F) = \{ w_1 d w_2 d \dots w_m d \mid w_i \text{ satisface } C_i \}$$

Para forzar que todas las interpretaciones coincidan:

$$w_1 = w_2 = \dots = w_m = w$$

Esto se logra intersectando con:

$$L_{0,1,d} = \{ (wd)^k \mid w \in \{0,1\}^*, k \geq 1 \}$$

Reducción de SAT

Reducción de SAT

SAT se resuelve determinando si el lenguaje:

$$L_{\text{sat}}(F) = L(\mathcal{A}_F) \cap L_{0,1,d}$$

es vacío.

SAT se resuelve determinando si el lenguaje:

$$L_{\text{sat}}(F) = L(\mathcal{A}_F) \cap L_{0,1,d}$$

es vacío.

Teorema

Si un formalismo es capaz de generar $L_{0,1,d}$ con una representación de tamaño constante, entonces no es posible que:

- sea cerrado bajo intersección con un DFA en tiempo polinomial, y

SAT se resuelve determinando si el lenguaje:

$$L_{\text{sat}}(F) = L(\mathcal{A}_F) \cap L_{0,1,d}$$

es vacío.

Teorema

Si un formalismo es capaz de generar $L_{0,1,d}$ con una representación de tamaño constante, entonces no es posible que:

- sea cerrado bajo intersección con un DFA en tiempo polinomial, y
 - se decida si es vacío en tiempo polinomial,
- simultáneamente. A menos que $P = NP$.

1 Preliminares

- Teoría de lenguajes
- Definición de SAT

2 Construcción del lenguaje de las interpretaciones satisfacibles

- Codificación de las interpretaciones como cadenas
- Autómata fórmula
- Recuperar la consistencia de las interpretaciones

3 Analizar las pMCFG

Gramáticas libres de contexto múltiples paralelas

Gramática usual

$S \rightarrow aSb$ permite hacer $aaSbb \Rightarrow aaaSbbb$

Gramática usual

$S \rightarrow aSb$ permite hacer $aaSbb \Rightarrow aaaSbbb$

- Los no terminales generan una **única cadena** $w \in T^*$.
- Cada derivación produce un solo resultado.

Gramáticas libres de contexto múltiples paralelas

Gramática usual

$S \rightarrow aSb$ permite hacer $aaSbb \Rightarrow aaaSbbb$

- Los no terminales generan una **única cadena** $w \in T^*$.
- Cada derivación produce un solo resultado.

Gramática libre de contexto múltiple paralela (pMCFG)

$$A \rightarrow f(B, C)$$

Gramáticas libres de contexto múltiples paralelas

Gramática usual

$S \rightarrow aSb$ permite hacer $aaSbb \Rightarrow aaaSbbb$

- Los no terminales generan una **única cadena** $w \in T^*$.
- Cada derivación produce un solo resultado.

Gramática libre de contexto múltiple paralela (pMCFG)

$$A \rightarrow f(B, C)$$

$$f[(x_1, x_3), (x_2)] = (x_1x_2, x_1x_3)$$

Gramáticas libres de contexto múltiples paralelas

Gramática usual

$S \rightarrow aSb$ permite hacer $aaSbb \Rightarrow aaaSbbb$

- Los no terminales generan una **única cadena** $w \in T^*$.
- Cada derivación produce un solo resultado.

Gramática libre de contexto múltiple paralela (pMCFG)

$A \rightarrow f(B, C)$

$f[(x_1, x_3), (x_2)] = (x_1x_2, x_1x_3)$

- Los no terminales generan **tuplas de cadenas** $(w_1, w_2, \dots, w_k)$.
- Las componentes se combinan mediante una **función de reordenamiento**.
- El símbolo inicial genera una tupla de un solo elemento.

Funciones de reordenamiento

Funciones de reordenamiento

Las funciones de reordenamiento determinan cómo se combinan las tuplas.

Las funciones de reordenamiento determinan cómo se combinan las tuplas.

Dos casos posibles

Las funciones de reordenamiento determinan cómo se combinan las tuplas.

Dos casos posibles

- **Lineales:** cada variable aparece a lo sumo una vez en la salida.

Las funciones de reordenamiento determinan cómo se combinan las tuplas.

Dos casos posibles

- **Lineales:** cada variable aparece a lo sumo una vez en la salida.
- **No lineales:** alguna variable puede repetirse en la salida.

Las funciones de reordenamiento determinan cómo se combinan las tuplas.

Dos casos posibles

- **Lineales:** cada variable aparece a lo sumo una vez en la salida.
- **No lineales:** alguna variable puede repetirse en la salida.

Caso especial

Si todas las funciones son lineales, la gramática se denomina:

MCFG (Gramática libre de contexto múltiple)

Intersección de pMCFG con DFA

Intersección de pMCFG con DFA

En el artículo de la introducción se propone un algoritmo para intersectar MCFG y pMCFG con DFA.

Intersección de pMCFG con DFA

En el artículo de la introducción se propone un algoritmo para intersectar MCFG y pMCFG con DFA.

$$(1) N' = \{S'\} \cup \{A[s_{10}, s_{11}, s_{20}, s_{21}, \dots, s_{d(A)0}, s_{d(A)1}] \mid A \in N, s_{ij} \in S_R, 1 \leq i \leq d(A), j = 0, 1\}.$$

(2.1) For each rule

$$A_0 \rightarrow f[A_1, A_2, \dots, A_{a(f)}]$$

in P and

$$A'_i = A_i[s_{10}^{(i)}, s_{11}^{(i)}, s_{20}^{(i)}, s_{21}^{(i)}, \dots, s_{d(A_i)0}^{(i)}, s_{d(A_i)1}^{(i)}], \quad 0 \leq i \leq a(f),$$

which satisfy the following *connecting condition*, let

$$A'_0 \rightarrow f[A'_1, A'_2, \dots, A'_{a(f)}] \in P'.$$

The connecting condition: let each component f^h ($1 \leq h \leq r(f) = d(A_0)$) of f be

$$f^h(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_{a(f)}) = \alpha_{h0} z_{h1} \alpha_{h1} z_{h2} \dots z_{hv_h(f)} \alpha_{hv_h(f)},$$

where

$$\bar{x}_i = (x_{i1}, x_{i2}, \dots, x_{id_i(f)}),$$

$$\alpha_{hk} \in T^*, \quad 0 \leq k \leq v_h(f),$$

$$z_{hk} = x_{i_{(h,k)} j_{(h,k)}}, \quad 1 \leq k \leq v_h(f), \quad 1 \leq i_{(h,k)} \leq a(f), \quad 1 \leq j_{(h,k)} \leq d_{i_{(h,k)}}(f).$$

Intersección de pMCFG con DFA

En el artículo de la introducción se propone un algoritmo para intersectar MCFG y pMCFG con DFA.

Then we have the following:

$$(a) \quad s_{j_{(h,1)}0}^{(i_{(h,1)})} = \sigma(s_{h0}^{(0)}, \alpha_{h0}),$$

$$(b) \quad s_{j_{(h,k)}0}^{(i_{(h,k)})} = \sigma(s_{j_{(h,k-1)}1}^{(i_{(h,k-1)})}, \alpha_{h(k-1)}), 2 \leq k \leq v_h(f), \text{ and}$$

$$(c) \quad s_{h1}^{(0)} = \sigma(s_{j_{(h,v_h(f))}1}^{(i_{(h,v_h(f))})}, \alpha_{hv_h(f)}).$$

$$(2.2) \quad S' \rightarrow S[s_0, s_F] \in P', s_F \in A_R.$$

(2.3) There is no rule except those mentioned above.

It is easily shown that

$$\begin{aligned} L_{G'}(A[s_{10}, s_{11}, s_{20}, s_{21}, \dots, s_{d(A)0}, s_{d(A)1}]) \\ = \{\bar{\alpha} = (\alpha_1, \alpha_2, \dots, \alpha_{d(A)}) \mid \bar{\alpha} \in L_G(A), s_{i1} = \sigma(s_{i0}, \alpha_i), 1 \leq i \leq d(A)\}. \end{aligned}$$

Hence, $L(G') = L \cap R$. $\square$

Intersección de pMCFG con DFA

En el artículo de la introducción se propone un algoritmo para intersectar MCFG y pMCFG con DFA.

$$s_{j(h,1)0}^{(i_{(h,1)})} = \sigma(s_{h0}^{(0)}, \alpha_{h0}),$$

$$s_{j(h,k)0}^{(i_{(h,k)})} = \sigma(s_{j(h,k-1)1}^{(i_{(h,k-1)})}, \alpha_{h(k-1)}),$$

$$s_{h1}^{(0)} = \sigma(s_{j(h,v_h(f))1}^{(i_{(h,v_h(f)})})}, \alpha_{hv_h(f)}).$$

Intersección de pMCFG con DFA

En el artículo de la introducción se propone un algoritmo para intersectar MCFG y pMCFG con DFA.

$$s_{j(h,1)0}^{(i_{(h,1)})} = \sigma(s_{h0}^{(0)}, \alpha_{h0}),$$

$$s_{j(h,k)0}^{(i_{(h,k)})} = \sigma(s_{j(h,k-1)1}^{(i_{(h,k-1)})}, \alpha_{h(k-1)}),$$

$$s_{h1}^{(0)} = \sigma(s_{j(h,v_h(f))1}^{(i_{(h,v_h(f))})}, \alpha_{hv_h(f)}).$$

...sencillo.

Idea del algoritmo

Idea del algoritmo

No terminal enriquecido

$$A[(p_1, q_1), \dots, (p_d, q_d)]$$

No terminal enriquecido

$$A[(p_1, q_1), \dots, (p_d, q_d)]$$

Producciones

$$A \rightarrow f(B, C) \quad f[(x_1, x_2), (x_3)] = (x_1 x_3, x_2)$$

Idea del algoritmo

No terminal enriquecido

$$A[(p_1, q_1), \dots, (p_d, q_d)]$$

Producciones

$$A \rightarrow f(B, C) \quad f[(x_1, x_2), (x_3)] = (x_1 x_3, x_2)$$

Invariante

$$(w_1, \dots, w_d) \in L(A[(p_1, q_1), \dots, (p_d, q_d)]) \iff (w_1, \dots, w_d) \in L(A)$$

y cada w_i lleva al DFA de p_i a q_i .

Idea del algoritmo

No terminal enriquecido

$$A[(p_1, q_1), \dots, (p_d, q_d)]$$

Producciones

$$A \rightarrow f(B, C) \quad f[(x_1, x_2), (x_3)] = (x_1 x_3, x_2)$$

Invariante

$$(w_1, \dots, w_d) \in L(A[(p_1, q_1), \dots, (p_d, q_d)]) \iff (w_1, \dots, w_d) \in L(A)$$

y cada w_i lleva al DFA de p_i a q_i .

Símbolo inicial

$$S' \rightarrow S[(q_0, q_f)] \quad q_f \in F_A$$

Consecuencia

La construcción es polinomial en el tamaño del DFA.

Complejidad de la construcción

Consecuencia

La construcción es polinomial en el tamaño del DFA.

Además

Las MCFG poseen un algoritmo polinomial para decidir si son vacías.

Contraejemplo

Contraejemplo

pMCFG

$$S \rightarrow f(A) \quad A \rightarrow g()$$

$$f[(x)] = xx \quad g() = (a)$$

Contraejemplo

pMCFG

$$S \rightarrow f(A) \quad A \rightarrow g()$$

$$f[(x)] = xx \quad g() = (a)$$

$$L(G) = \{aa\}$$

Contraejemplo

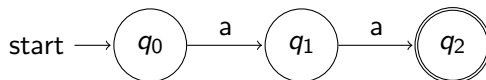
pMCFG

$$S \rightarrow f(A) \quad A \rightarrow g()$$

$$f[(x)] = xx \quad g() = (a)$$

$$L(G) = \{aa\}$$

DFA que reconoce $\{aa\}$



Contraejemplo

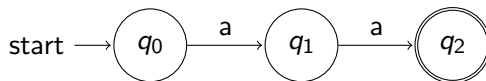
pMCFG

$$S \rightarrow f(A) \quad A \rightarrow g()$$

$$f[(x)] = xx \quad g() = (a)$$

$$L(G) = \{aa\}$$

DFA que reconoce $\{aa\}$



$$L(G) \cap L(\mathcal{A}) = \{aa\}$$

¿Qué falla?

¿Qué falla?

La producción principal es

$$S \rightarrow f(A) \quad f(x) = xx$$

¿Qué falla?

La producción principal es

$$S \rightarrow f(A) \quad f(x) = xx$$

Para reconocer la cadena aa :

$$q_0 \xrightarrow{x} q_1 \quad q_1 \xrightarrow{x} q_2$$

¿Qué falla?

La producción principal es

$$S \rightarrow f(A) \quad f(x) = xx$$

Para reconocer la cadena aa :

$$q_0 \xrightarrow{x} q_1 \quad q_1 \xrightarrow{x} q_2$$

Sin embargo, el algoritmo asigna un único par de estados a cada variable:

$$A[(p, q)] \quad (q_0, q_1) \neq (q_1, q_2)$$

¿Qué falla?

La producción principal es

$$S \rightarrow f(A) \quad f(x) = xx$$

Para reconocer la cadena aa :

$$q_0 \xrightarrow{x} q_1 \quad q_1 \xrightarrow{x} q_2$$

Sin embargo, el algoritmo asigna un único par de estados a cada variable:

$$A[(p, q)] \quad (q_0, q_1) \neq (q_1, q_2)$$

Consecuencia

Las dos ocurrencias de x necesitan pares de estados distintos.

La invariante del algoritmo deja de cumplirse.

$$L_{0,1,d} = \{(wd)^n \mid w \in \{0,1\}^*, n \geq 1\}$$

$$L_{0,1,d} = \{(wd)^n \mid w \in \{0,1\}^*, n \geq 1\}$$

Es posible construir una pMCFG $G_{0,1,d}$ para él, donde $N = \{S, C, A\}$ y $T = \{0, 1, d\}$. Se tiene que P y F contienen las siguientes producciones y funciones respectivamente:

$$S \rightarrow res(C)$$

$$res[(x, y)] = (y)$$

$$C \rightarrow copy(C)$$

$$copy[(x, y)] = (x, xy)$$

$$C \rightarrow copy(A)$$

$$A \rightarrow f_0(A)$$

$$f_0[(x, y)] = (0x, y)$$

$$A \rightarrow f_1(A)$$

$$f_1[(x, y)] = (1x, y)$$

$$A \rightarrow g()$$

$$g() = (d, \epsilon)$$

$$L_{0,1,d} = \{(wd)^n \mid w \in \{0,1\}^*, n \geq 1\}$$

Es posible construir una pMCFG $G_{0,1,d}$ para él, donde $N = \{S, C, A\}$ y $T = \{0, 1, d\}$. Se tiene que P y F contienen las siguientes producciones y funciones respectivamente:

$$S \rightarrow \text{res}(C)$$

$$\text{res}[(x, y)] = (y)$$

$$C \rightarrow \text{copy}(C)$$

$$\text{copy}[(x, y)] = (x, xy)$$

$$C \rightarrow \text{copy}(A)$$

$$A \rightarrow f_0(A)$$

$$f_0[(x, y)] = (0x, y)$$

$$A \rightarrow f_1(A)$$

$$f_1[(x, y)] = (1x, y)$$

$$A \rightarrow g()$$

$$g() = (d, \epsilon)$$

Resultado

Existe una pMCFG de tamaño constante $G_{0,1,d}$ que genera $L_{0,1,d}$.

Como no se puede tener simultáneamente que las pMCFG:

- sean cerradas bajo intersección con un DFA en tiempo polinomial, y

Como no se puede tener simultáneamente que las pMCFG:

- sean cerradas bajo intersección con un DFA en tiempo polinomial, y
- se decida si son vacías en tiempo polinomial,

y el problema de vacío para pMCFG es decidible en tiempo polinomial necesariamente...

Como no se puede tener simultáneamente que las pMCFG:

- sean cerradas bajo intersección con un DFA en tiempo polinomial, y
- se decida si son vacías en tiempo polinomial,

y el problema de vacío para pMCFG es decidible en tiempo polinomial necesariamente...

Teorema

No existe un algoritmo polinomial que construya una pMCFG para la intersección de una pMCFG con un DFA a menos que $P = NP$.

Teorema general

Si un formalismo es capaz de generar $L_{0,1,d}$ con una representación de tamaño constante, entonces no es posible que:

- sea cerrado bajo intersección con DFA mediante una construcción polinomial,
 - y que el problema de vacío sea decidible en tiempo polinomial,
- simultáneamente, a menos que $P = NP$.

Teorema general

Si un formalismo es capaz de generar $L_{0,1,d}$ con una representación de tamaño constante, entonces no es posible que:

- sea cerrado bajo intersección con DFA mediante una construcción polinomial,
 - y que el problema de vacío sea decidable en tiempo polinomial,
- simultáneamente, a menos que $P = NP$.

Aplicación a las pMCFG

No existe un algoritmo polinomial que construya una pMCFG para $L(G) \cap L(\mathcal{A})$ dada una pMCFG G y un DFA $\mathcal{A}$, a menos que $P = NP$

- Estudiar formalismos capaces de generar $L_{0,1,d}$.

Recomendaciones y trabajo futuro

- Estudiar formalismos capaces de generar $L_{0,1,d}$.
- Demostrar o refutar que toda representación de $L_{0,1,d}$ tiene tamaño constante.

Recomendaciones y trabajo futuro

- Estudiar formalismos capaces de generar $L_{0,1,d}$.
- Demostrar o refutar que toda representación de $L_{0,1,d}$ tiene tamaño constante.
- Investigar si las gramáticas libres de contexto múltiples paralelas son cerradas bajo intersección con lenguajes regulares.

- Estudiar formalismos capaces de generar $L_{0,1,d}$.
- Demostrar o refutar que toda representación de $L_{0,1,d}$ tiene tamaño constante.
- Investigar si las gramáticas libres de contexto múltiples paralelas son cerradas bajo intersección con lenguajes regulares.
- Analizar la complejidad del autómata para una fórmula en el caso en el que la fórmula está en 3-CNF.

- Demuestre o refute que $P = NP$.

El problema de la satisfacibilidad booleana y el vacío de la intersección con lenguajes regulares

Jossué Arteché Muñoz

Facultad de Matemática y Computación
Universidad de La Habana

10 de junio del 2026

Tutores: MSc. Fernando Raul Rodriguez Flores
Lic. Raudel Alejandro Gómez Molina

- Analice las consecuencias de la refutación presentada para la literatura que utiliza el cierre de las pMCFG bajo intersección con lenguajes regulares y/o el algoritmo de construcción asociado. Distinga entre los usos del resultado para pMCFG generales, potencialmente no lineales, y los usos restringidos a MCFG, LCFRS u otros formalismos lineales. Indique cuáles resultados posteriores podrían quedar sin demostración, cuáles requerirían una corrección, y en qué casos no se afectan las conclusiones obtenidas.

En el artículo “Finite state translation systems and parallel multiple context-free grammars” se utiliza que las pMCFG son cerradas bajo intersección con lenguajes regulares, para demostrar que el lenguaje generado por los sistemas de traducción de estado finito deterministas es exactamente el mismo que el generado por las pMCFG.

- Explique, usando la presentación original de Seki et al., qué papel cumple la no linealidad de las funciones de reordenamiento en el contraejemplo presentado. En particular, identifique el paso de la construcción del Teorema 3.9(3) donde se pierde la correspondencia esperada con el lenguaje intersección y discuta si el mismo tipo de fallo podría aparecer, o no, en el caso restringido de las MCFG.

It is easily shown that

$$\begin{aligned} &L_{G'}(A[s_{10}, s_{11}, s_{20}, s_{21}, \dots, s_{d(A)0}, s_{d(A)1}]) \\ &= \{\tilde{\alpha} = (\alpha_1, \alpha_2, \dots, \alpha_{d(A)}) \mid \tilde{\alpha} \in L_G(A), s_{i1} = \sigma(s_{i0}, \alpha_i), 1 \leq i \leq d(A)\}. \end{aligned}$$

Hence, $L(G') = L \cap R$. $\square$

Se deja de cumplir la invariante

$$(w_1, \dots, w_d) \in L(A[(p_1, q_1), \dots, (p_d, q_d)]) \iff (w_1, \dots, w_d) \in L(A)$$

y cada w_i lleva al DFA de p_i a q_i .

Se deja de cumplir la invariante

$$(w_1, \dots, w_d) \in L(A[(p_1, q_1), \dots, (p_d, q_d)]) \iff (w_1, \dots, w_d) \in L(A)$$

y cada w_i lleva al DFA de p_i a q_i .

Símbolo inicial

$$S' \rightarrow S[(q_0, q_f)] \quad q_f \in F_A$$

- Comente posibles restricciones o modificaciones bajo las cuales podría recuperarse la correctitud de la construcción de intersección entre una pMCFG y un DFA. En particular, valore si basta restringirse al caso lineal de las MCFG, si podría admitirse alguna forma limitada de copia, o si sería necesario cambiar sustancialmente la información que la construcción mantiene sobre los estados del autómata.

Enriquecimiento

Cada no terminal A de dimensión d se decora con una tupla de *conjuntos* de pares de estados:

$$A[R_1, \dots, R_d], \quad R_i \subseteq Q_A \times Q_A.$$

Enriquecimiento

Cada no terminal A de dimensión d se decora con una tupla de *conjuntos* de pares de estados:

$$A[R_1, \dots, R_d], \quad R_i \subseteq Q_A \times Q_A.$$

Invariante

$$(w_1, \dots, w_d) \in L(A[R_1, \dots, R_d]) \iff (w_1, \dots, w_d) \in L(A)$$

y cada w_i lleva al DFA de p a q para cada $(p, q) \in R_i$.

Enriquecimiento

Cada no terminal A de dimensión d se decora con una tupla de *conjuntos* de pares de estados:

$$A[R_1, \dots, R_d], \quad R_i \subseteq Q_A \times Q_A.$$

Invariante

$$(w_1, \dots, w_d) \in L(A[R_1, \dots, R_d]) \iff (w_1, \dots, w_d) \in L(A)$$

y cada w_i lleva al DFA de p a q para cada $(p, q) \in R_i$.

Símbolo inicial

$$S' \rightarrow S[\{(q_0, q_f)\}] \quad q_f \in F_A$$

- Valore si, a partir de una instancia F de SAT con m cláusulas, puede construirse una MCFG que genere el lenguaje $\{(wd)^m \mid w \in \{0,1\}^*\}$, de modo que al intersectarla con el lenguaje regular $L_{ind}(F)$ se obtenga una reducción alternativa basada solo en MCFG, caso que queda fuera del contraejemplo presentado. Discuta, en particular, qué ocurre con la dimensión de la gramática, con el tamaño de la construcción de intersección resultante y si este enfoque es útil o teóricamente interesante desde el punto de vista de la complejidad de SAT.

La *dimensión* (fan-out) de un no terminal es el número de componentes de las tuplas que deriva.

La *dimensión* (fan-out) de un no terminal es el número de componentes de las tuplas que deriva.

Definición

Una k -MCFG es una MCFG en la que ningún no terminal tiene dimensión mayor que k . Su clase de lenguajes es la de los k -MCFL.

Costo de la intersección con un DFA

Decorar cada no terminal de dimensión d con d pares de estados produce $\sim |Q_{\mathcal{A}}|^{2d}$ instancias.

Costo de la intersección con un DFA

Decorar cada no terminal de dimensión d con d pares de estados produce $\sim |Q_{\mathcal{A}}|^{2d}$ instancias.

Polinomial en los estados $|Q_{\mathcal{A}}|$, **exponencial en la dimensión** de la gramática.

Lema

Para todo k -MCFL L infinito existen $u_j \in T^*$ ($1 \leq j \leq k+1$) y $v_j, w_j, s_j \in T^*$ ($1 \leq j \leq k$) tales que

① $\sum_{j=1}^k |v_j s_j| > 0$, y

② para todo $i \geq 0$,

$$z_i = u_1 v_1^i w_1 s_1^i u_2 v_2^i w_2 s_2^i \cdots u_k v_k^i w_k s_k^i u_{k+1} \in L.$$

Por qué $L_{0,1,d}$ no es k -MCFL para ningún k fijo

Al aplicar el teorema se tiene que

$$L_k \text{ es } k\text{-MCFL} \quad \text{pero} \quad L_k \notin (k-1)\text{-MCFL}.$$

Por qué $L_{0,1,d}$ no es k -MCFL para ningún k fijo

Al aplicar el teorema se tiene que

$$L_k \text{ es } k\text{-MCFL} \quad \text{pero} \quad L_k \notin (k-1)\text{-MCFL}.$$

Luego $L_{0,1,d} = \bigcup_{k \geq 1} L_k$ no es k -MCFL para ningún k fijo.

Por qué $L_{0,1,d}$ no es k -MCFL para ningún k fijo

Al aplicar el teorema se tiene que

$$L_k \text{ es } k\text{-MCFL} \quad \text{pero} \quad L_k \notin (k-1)\text{-MCFL}.$$

Luego $L_{0,1,d} = \bigcup_{k \geq 1} L_k$ no es k -MCFL para ningún k fijo.

Consecuencia

Más cláusulas $\Rightarrow$ más copias $\Rightarrow$ mayor dimensión, y la intersección con un regular crece **exponencialmente** en la dimensión.